Ch 09: Dictionaries

Application Program Development

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2026



EAST TEXAS A&M

Dictionaries

Dictionary

A dictionary is an object that stores a collection of data. Each element in a dictionary has two parts: a key and a value. You use a key to locate a specific value.

- **Dictionary**: object that stores a collection of key/value pairs
  - Each element consists of a **key** and a **value**
    - Referred to as: map, associative array, hash, key-value pair, Dictionary, indexed table, etc. (Wikipedia, 2026)
    - key must be an immutable object (e.g. string, int, float, tuple, **not** a list)
  - To retrieve a specific value, need to know and use the key associated with it
  - Access by key is constant or *logN* time depending on implementation, much faster than sequential access

Format for creating a dictionary:

```
phonebook = {'Chris': '555-1111',
             'Katie': '555-2222',
             'Joanne': '555-3333'}
```

EAST TEXAS A&M

Retrieving a Value from a Dictionary

- Elements in dictionary are unsorted
  - In fact they are indexed (hashed or a tree) by the key

General format for retrieving value from dictionary:

```
dictionary[key]
number = phonebook['Chris']
```

- if `key` is in the dictionary, associated value is returned
  - if not, a `KeyError` exception is raised

- To test whether a `key` is in a dictionary use the `in` and `not in` operators
  - Helps prevent `KeyError` exceptions

```
if key in phonebook:
    print(f'{key} number is {phonebook[key]}')
else:
    print(f'I do not know {key}')
```

EAST TEXAS A&M

Adding Elements to an Existing Dictionary

- Dictionaries are **mutable** objects
- To add a new key-value pair, simply assign a value to a key:

```
dictionary[key] = value
phonebook['Joe'] = '555-0123'
```

- If key exists in the dictionary, the value associated with the existing key will be changed (e.g. this is a way to modify a record)

```
# modifies phone number if `Chris`
# already exists
phonebook['Chris'] = '555-4444'
```



Deleting Elements from an Existing Dictionary

- To delete a key-value pair, use the `del` keyword

```
del dictionary[key]
del phonebook['Chris']
```

- If `key` is not in the dictionary, `KeyError` exception is raised
  - So usually check using `key in dictionary` before deleting

EAST TEXAS A&M

Getting the Number of Elements and Mixing Data Types

- `len` function: used to obtain number of elements in a dictionary

```
num_items = len(phonebook)
```

- `keys` must be immutable objects, but associated values can be any type of object
  - One dictionary can include keys of several different immutable types
    - Though this is an unusual use case
  - Values stored in a single dictionary can be of different types
    - Though also this is not a typical thing to do



Creating an Empty Dictionary and Using a `for` Loop to Iterate Over a Dictionary

- To create an empty dictionary:
  - use {}
  - Use built-in function `dict()`
  - Elements can be added to dictionary as program executes

```
# both are commonly seen
phonebook = {}
phonebook = dict()
```

- Use a for loop to iterate over a dictionary

```
for name in phonebook:
    print(f'name: {name} num: {phonebook[name]}')
```



(Some) Dictionary Methods

Method	Description
`clear()`	Clears the contents of a dictionary
`get()`	Gets the value associated with a specified key, returns default if key not found
`items()`	Returns all the keys in a dict and their values as a sequence of tuples
`keys()`	Returns all the keys in a dictionary as a tuple
`pop()`	Returns the value associated with a key and removes the key-value pair from the dict
`popitem()`	Returns, as a tuple, the last added key-value pair and removes it.
`values()`	Returns all the values in dict as a tuple

See Python Standard Library Documentation (Python, 2026)



Dictionary Comprehensions (1 of 2)

Dictionary Comprehension

A **dictionary comprehension** is an expression that reads a sequence of input elements and uses those input elements to produce a dictionary.
(Same idea as a list comprehension)

Using a `for` loop:

```
numbers = [1, 2, 3, 4]
squares = {}
for item in numbers:
    squares[item] = item**2
```

Using a dictionary comprehension:

```
squares = {item: item**2 for item in numbers}
```



Dictionary Comprehensions (2 of 2)

Example: create dictionary of string keys and their lengths as values

```
names = ['Jeremy', 'Kate', 'Peg']
str_lengths = {item: len(item) for item in names}
print(str_lengths)
```

```
{'Jeremy': 6, 'Kate': 4, 'Peg': 3}
```

Example: making a copy of a dictionary

```
dict1 = {'A': 1, 'B': 2, 'C': 3}
dict2 = {k:v for k,v in dict1.items()}
print(dict2)
```

```
{'A': 1, 'B': 2, 'C': 3}
```



Summary

- A dictionary is collection that stores key-value pairs
  - You can lookup items using the dictionary key
  - You can add or modify items in a dictionary by (re)assigning the key to a value
  - You can use the `in` and `not in` operators to test if a `key` is present or not
  - You can use the `del` keyword to remove key-value pairs from a dictionary
- Dictionaries are collection objects that have a lot of useful member methods
  - Among these are `get()`, `items()`, `keys()`, `values()`
- Dictionary comprehensions are a part of the syntax, and work similar to list comprehensions



EAST TEXAS A&M

References

- Python. (2026). *Python 3 standard library documentation*. Retrieved August 13, 2026, from https://docs.python.org/3/
- Wikipedia. (2026). *Associative array (computer science)*. Retrieved August 14, 2026, from https://en.wikipedia.org/wiki/Associative_array

